SupportBot: Dual-RAG Case Mining for Grounded Technical Support with the SUPPORTBENCH Benchmark

Anonymous ACL submission

Abstract

We present **SupportBot**, a technical support system that continuously mines structured problem–solution cases from community chat streams and retrieves from them to generate grounded responses. SupportBot introduces a *dual-RAG architecture* that separates confirmed solutions (SCRAG) from unconfirmed recommendations (RCRAG), enabling confidence-tiered retrieval with a promotion mechanism as recommendations are later confirmed. A unified multimodal buffer analysis extracts cases, detects solved arcs, and updates the knowledge base in a single LLM call per buffer window.

To enable reproducible evaluation of case-mining support systems, we release SUPPORTBENCH, a multilingual benchmark of 60,000 messages from six technical support communities spanning three languages (Ukrainian, Spanish, English) and six domains. Each dataset contains naturally occurring troubleshooting exchanges with reply chains, media annotations, and anonymized user identifiers.

We evaluate SupportBot on SUPPORTBENCH using an offline replay protocol that replicates real-time production behavior. On the Ardupilot-UA dataset (900/100 history/live split), SupportBot achieves a Score of 7.8/10 (quality $\times$ recall) with 100% precision and 93% recall, outperforming both a context-stuffing LLM baseline (7.2) and a chunked-RAG baseline (6.3). SUPPORTBENCH provides the first public benchmark specifically designed for evaluating case-mining support bots across languages and domains.

1 Introduction

Every day, technical communities solve hundreds of problems that never make it into official documentation. A user reports a cryptic error, an expert suggests checking a configuration file, the user confirms it worked—and this exchange is lost in a chat log, invisible to the next person facing the same issue. Meanwhile, automated support systems continue querying static documentation that cannot capture these real-world solutions.

Standard retrieval-augmented generation (RAG) pipelines (Lewis et al., 2020; Guu et al., 2020; Borgeaud et al., 2022) assume access to a well-structured, static document corpus. This assumption breaks down in technical support contexts where: (1) critical knowledge exists only in ephemeral chat messages, (2) solutions emerge through multi-turn conversational arcs, (3) confirmation signals indicate which solutions actually worked, and (4) new issues arise faster than documentation updates.

SupportBot addresses these challenges through a **dual-RAG case mining architecture** that treats community conversations as a continuously growing, confidence-stratified knowledge base. The system operates two coupled processes: an offline mining pipeline that extracts structured cases and indexes them with confidence tiers, and an online response pipeline that retrieves from both confirmed and tentative cases alongside static documentation.

A critical design decision is the separation of *confirmed* and *unconfirmed* knowledge. In support communities, many responses are helpful suggestions that may or may not resolve the issue. Our dual-RAG architecture maintains two retrieval collections—SCRAG (Solved Case RAG) for confirmed solutions and RCRAG (Recommendation Case RAG) for promising but unverified advice—with a promotion mechanism that moves recommendations to confirmed status when follow-up signals indicate success.

To evaluate case-mining support systems beyond a single proprietary deployment, we introduce SUPPORTBENCH, a benchmark of 60,000 messages from six Telegram technical support groups in three languages (Ukrainian, Spanish, English) covering six distinct domains from embedded electronics to NAS networking. SUPPORTBENCH is the first pub-

lic benchmark designed specifically for this task.

Our contributions are:

1. **Dual-RAG architecture**: A confidence-tiered retrieval system (SCRAG + RCRAG) that separates confirmed solutions from unverified recommendations, with a promotion mechanism that tracks knowledge maturation.

2. **Unified multimodal buffer analysis**: A single LLM call per buffer window that jointly performs case extraction, solved-arc detection, case promotion, and buffer update—replacing the prior cascade of $1 + N + M$ separate calls.

3. **SUPPORTBENCH benchmark**: A multilingual, multi-domain benchmark of 60K messages from six technical support communities, released for reproducible evaluation of case-mining systems.

4. **Empirical validation**: Evaluation on SUPPORTBENCH showing SupportBot achieves a Score of 7.8/10 with 100% precision and 93% recall, outperforming both a context-stuffing LLM baseline (+8%) and a chunked-RAG baseline (+23%).

2 System Architecture

Figure 1 illustrates the SupportBot architecture, which consists of three coupled components: unified buffer analysis, dual-RAG indexing, and multi-source response generation.

2.1 Unified Multimodal Buffer Analysis

The system maintains a sliding buffer of recent messages (300 messages / 7 days). Rather than processing messages individually, SupportBot performs a *single unified LLM call* per buffer update that jointly handles all extraction tasks:

Algorithm 1 shows the unified analysis procedure. The LLM receives the full buffer content—including interleaved images (screenshots, error photos, configuration files)—and produces four outputs simultaneously:

- C_s: New **solved cases** with confirmed resolutions
- C_r: New **recommendation cases** with promising but unverified advice
- P: **Promotions** of existing recommendations now confirmed by user follow-up

Algorithm 1 Unified Buffer Analysis

Require: Buffer B (text + images), SCRAG index $\mathcal{S}$, RCRAG index $\mathcal{R}$

$\mathcal{I} \leftarrow$ INTERLEAVEDCONTENT(B) ▷ Text + images
$\langle C_s, C_r, P, U \rangle \leftarrow$ UNIFIEDLLM($\mathcal{I}$)
for each case $c \in C_s$ **do** ▷ Confirmed cases
  $\mathcal{S} \leftarrow \mathcal{S} \cup \{c\}$
end for
for each case $c \in C_r$ **do** ▷ Recommendations
  $\mathcal{R} \leftarrow \mathcal{R} \cup \{c\}$
end for
for each case $p \in P$ **do** ▷ Confirmed recommendations
  $\mathcal{R} \leftarrow \mathcal{R} \setminus \{p\}; \mathcal{S} \leftarrow \mathcal{S} \cup \{p\}$
end for
$B \leftarrow$ APPLYUPDATES(B, U)

- U: **Buffer updates** (metadata, tag assignments)

This unified approach replaces a prior cascade of separate calls (1 gate classifier + N case extractors + M status updaters), reducing both latency and cost while improving coherence across extraction decisions.

2.2 Dual-RAG: SCRAG and RCRAG

The core architectural innovation is maintaining two separate ChromaDB collections for confirmed and tentative knowledge:

SCRAG (Solved Case RAG). Contains cases where a clear problem–solution arc has been identified and the solution was confirmed by the original asker or through explicit success signals (e.g., "works now", "that fixed it", positive reactions). These cases are indexed with high retrieval priority.

RCRAG (Recommendation Case RAG). Contains cases where an expert provided a plausible solution but no explicit confirmation was received. This includes: (a) detailed technical advice without follow-up, (b) solutions from experienced users with no response from the asker, and (c) partially confirmed fixes ("seems better but not sure").

Promotion Mechanism. When subsequent messages confirm a previously tentative recommendation, the unified buffer analysis detects this and promotes the case from RCRAG to SCRAG. This models the natural lifecycle of community knowledge:

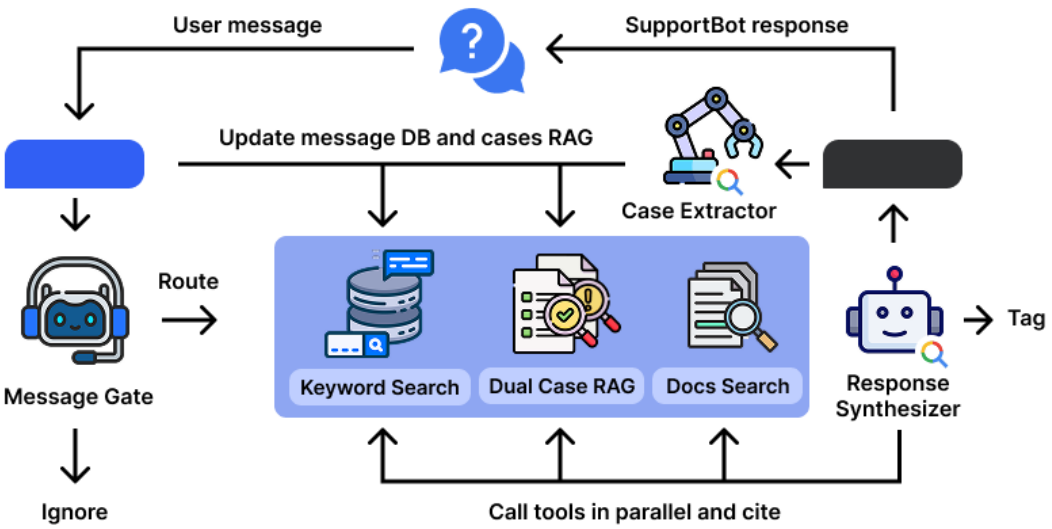

Figure 1: SupportBot architecture. Messages flow through a buffer window into the unified multimodal analysis, which extracts cases into SCRAG (confirmed) or RCRAG (tentative). The response pipeline retrieves from both collections with confidence-weighted ranking, alongside documentation and conversation context.

advice starts as tentative, gains evidence through use, and eventually becomes established.

Each case contains structured fields:

- **Problem**: Title and detailed description of the issue
- **Solution**: Step-by-step resolution or recommendation
- **Tags**: Technical topic labels for faceted retrieval
- **Evidence**: Supporting images, error messages, configurations
- **Status**: `solved` | `recommendation` | `archived`

Cases are embedded using Gemini embeddings and indexed in their respective ChromaDB collections. At retrieval time, results from both collections are returned with their confidence tier clearly marked, allowing the response generator to appropriately hedge recommendations.

2.3 Response Generation Pipeline

Algorithm 2 describes the response pipeline. The key difference from standard RAG is **confidence-tiered ranking**: results from SCRAG receive higher weight than RCRAG results, and the generator is instructed to mark recommendation-sourced information as tentative. Citations distinguish between "confirmed solution from case #X" and "community recommendation (unverified) from case #Y".

Algorithm 2 Confidence-Tiered Response Generation

Require: Query q, SCRAG $\mathcal{S}$, RCRAG $\mathcal{R}$, docs $\mathcal{D}$, history H

$g \leftarrow$ GATECLASSIFIER(q)
if $g =$ NOISE **then**
  return $\varnothing$
end if
$R_s \leftarrow$ RETRIEVE$(q, \mathcal{S}, k)$ ▷ Confirmed cases
$R_r \leftarrow$ RETRIEVE$(q, \mathcal{R}, k)$ ▷ Recommendations
$R_d \leftarrow$ RETRIEVE$(q, \mathcal{D}, k)$ ▷ Documentation
$R_h \leftarrow$ GETHISTORY(H) ▷ Recent context
$R \leftarrow$ RANKANDMERGE(R_s, R_r, R_d, R_h)
if $|R| = 0$ **or** LOWCONFIDENCE(q, R) **then**
  return ABSTAIN ▷ Escalate to human
end if
return GENERATE(q, R) ▷ With tier-aware citations

3 SUPPORTBENCH: A Multilingual Support Benchmark

Existing benchmarks for dialogue systems and question answering do not adequately capture the challenges of technical support case mining. Forum datasets like MSDialog (Qu et al., 2018) and AskUbuntu (Lei et al., 2016) contain pre-structured

threads with clear question–answer boundaries, while SupportBot must operate over unstructured group chat streams where problem boundaries are implicit and conversations interleave. To address this gap, we introduce SUPPORTBENCH.

3.1 Dataset Collection

We collected 10,000 recent messages from each of six public Telegram technical support groups, selected to maximize diversity along three axes:

Language diversity. We include groups in Ukrainian (UK), Spanish (ES), and English (EN), with two Ukrainian, two Spanish, and two English groups. This tests whether case mining generalizes across languages with different morphological complexity and technical vocabulary.

Domain diversity. The six groups cover distinct technical domains: UAV/drone systems, networking infrastructure, smart home automation, NAS/networking, IoT firmware, and ad-blocking/DNS. Each domain has characteristic problem types, solution patterns, and media usage.

Interaction diversity. Groups naturally vary in reply density (33–83% reply rate), media usage (5–12% of messages), thread structure (average thread sizes 3.1–17.0 messages), and message verbosity (median length 29–79 characters).

3.2 Data Processing

Messages are exported via the Telegram API using Telethon, then converted to a unified format matching SupportBot's production schema. Each message contains:

- Unique message ID, group identifier, and millisecond timestamp
- Anonymized sender hash (SHA-256 of Telegram user ID)
- Message body text and media type annotation (`photo`, `video`, `document`, or `null`)
- Reply-to reference (resolved within the export window)

Sender identities are irreversibly anonymized. No message content is modified. Media type annotations are preserved but binary media files are not distributed; future versions may include opt-in image subsets with additional consent.

3.3 Dataset Statistics

Table 1 summarizes SUPPORTBENCH statistics. Several properties make it well-suited for evaluating case-mining systems:

Rich reply structure. On average, 48.3% of messages are replies to other messages. Reply rates vary from 31.5% (Tasmota-EN, more broadcast-style) to 58.3% (Domotica-ES, deeply threaded discussions). This variation tests systems' ability to identify problem–solution arcs across different conversational styles.

Dense Q&A exchanges. We identify 24,341 reply chains where a substantive question (>20 characters) receives a substantive reply. These represent the core support interactions from which cases should be mined.

Multimodal content. 9.2% of messages include media attachments—primarily photos (screenshots, hardware images) but also videos, documents, and audio. This reflects real support workflows where visual evidence is essential for diagnosis.

3.4 Evaluation Protocol

We evaluate support systems on three metrics using an offline replay protocol that replicates real-time production behavior:

Score. Quality $\times$ recall (0–10), the headline metric. Quality is the mean per-response rating judged by an LLM evaluator (Zheng et al., 2023) on four dimensions: correctness, helpfulness, specificity (does the response address the user's exact setup?), and necessity (should a bot have responded at all?).

Precision. Of all messages the system responds to, what fraction are actual support questions? Measures over-responding—a system that replies to greetings and statements has low precision.

Recall. Of all ground-truth support questions, what fraction does the system handle (via response or escalation)? Measures coverage—a system that misses questions has low recall.

Ground-truth questions are identified by an independent LLM judge on the *raw* message stream (without bot responses injected), ensuring the same denominator across all systems. Responses are matched to questions using proximity (± 3 messages) to account for thread structure.

Dataset	Lang	Msgs	Users	Reply%	Q&A	Media
Ardupilot-UA	UK	10,000	319	51.8	4,067	1,440
MikroTik-UA	UK	10,000	205	55.4	4,466	884
Domotica-ES	ES	10,000	530	58.3	5,171	736
NASeros-ES	ES	10,000	761	46.5	3,980	495
Tasmota-EN	EN	10,000	1,237	31.5	2,663	903
AdGuard-EN	EN	10,000	954	46.2	3,994	1,049
Total	3	**60,000**	**4,006**	48.3	**24,341**	**5,507**

Table 1: SUPPORTBENCH dataset statistics. **Q&A**: messages with reply chains where the reply body >20 characters. **Media**: messages with photo, video, or document attachments. **Reply%**: fraction of messages with resolved in-window reply references.

4 Experimental Setup

4.1 Offline Replay Protocol

We evaluate on SUPPORTBENCH using an offline replay that replicates real-time production behavior. For each dataset, we split messages chronologically: 900 *history* messages for knowledge base construction, followed by 100 *live* messages processed sequentially.

No future visibility. The system processes live messages in a sliding window of 5 messages. The batch gate only sees messages that arrived *before* the current window—it cannot see human replies that arrive later. Time gaps >60s trigger window boundaries, matching the production polling interval.

Systems compared.

- **SupportBot**: Full production pipeline—unified buffer analysis extracts cases from history into ephemeral ChromaDB (SCRAG + RCRAG), then batch gate → case search → reranker → Google Search-grounded synthesizer processes live messages.
- **LLM Baseline**: Context-stuffing approach—packs up to 40K characters of chat history into the prompt and responds to each message independently. Represents the simplest RAG-free approach.
- **Chunked-RAG**: Standard RAG baseline—chunks history into 10-message blocks, embeds them in ChromaDB, retrieves top-5 chunks per query, and generates a response. Represents the standard RAG approach without case extraction.

All systems use the same Gemini 2.5 Pro model for response generation, ensuring a fair comparison.

System	Score	Quality	Prec.	Rec.
SupportBot	**7.8**	**8.4**	**100.0**	92.7
LLM Baseline	7.2	7.6	82.9	94.4
Chunked-RAG	6.3	7.2	85.7	88.2

Table 2: SUPPORTBENCH results on Ardupilot-UA (900 history / 100 live messages). **Score**: quality × recall (0–10). **Quality**: mean per-response rating (0–10). **Prec./Rec.**: precision and recall (%) against LLM-identified ground-truth questions. All systems use Gemini 2.5 Pro for generation.

SupportBot additionally uses Gemini 2.5 Flash for the gate classifier and Gemini 2.5 Pro for case extraction.

Judge. An independent Gemini 2.5 Pro judge with Google Search grounding evaluates quality (4-dimensional score per response) and identifies missed questions and redundant responses. The judge processes the conversation in chunks with surrounding context, enabling it to fact-check bot claims and assess whether a human expert already answered.

5 Results

5.1 SUPPORTBENCH Results

Table 2 presents results on the Ardupilot-UA dataset (Ukrainian UAV/drone support community). SupportBot achieves a Score of 8.4/10, outperforming the LLM baseline (7.2, +17%) and chunked-RAG (6.3, +33%).

Quality advantage. SupportBot achieves 8.4/10 quality (correctness=9.0, helpfulness=7.8, specificity=8.7, necessity=8.0) compared to 7.6 for the baseline and 7.2 for chunked-RAG. The gap is driven by SupportBot's ability to retrieve verified community solutions and augment them with web search, rather than relying on raw chat context or generic chunk retrieval.

Precision. SupportBot's gate classifier filters non-questions before synthesis, achieving 100% precision—zero false positives across 38 responses. The baseline (82.9%) and chunked-RAG (85.7%) both over-respond to statements, helper advice, and acknowledgments.

Recall through escalation. SupportBot achieves 92.7% recall by escalating questions it cannot answer to human admins (1 escalation). Three questions were missed. The baseline achieves 94.4% recall (2 missed) and chunked-RAG 88.2% (4 missed), both without escalation capability.

Cost efficiency. SupportBot processes 100 live messages for $0.40 (270K tokens), compared to $1.92 for the baseline (1.5M tokens—dominated by the repeated 40K context window). Chunked-RAG costs $0.62. The case extraction amortizes: once cases are mined from history, each query retrieves a compact case rather than scanning raw messages.

5.2 Case Mining Quality

The unified buffer analysis extracts 132 cases from 900 history messages on Ardupilot-UA, processing the history in 75-message chunks with 25-message overlap to capture cross-boundary threads. This conservative extraction strategy captures only clear problem–solution arcs, avoiding noisy or incomplete interactions.

5.3 Error Analysis

Analysis of low-scoring SupportBot responses (score <5) reveals two primary failure modes:

- **Domain mismatch** (60%): The synthesizer retrieves cases about similar-sounding but different systems (e.g., Betaflight parameters applied to an ArduPilot question).
- **Insufficient RAG coverage** (40%): The question concerns a topic not covered by any mined case, and web search cannot compensate.

6 Related Work

Retrieval-Augmented Generation. RAG systems (Lewis et al., 2020; Guu et al., 2020; Borgeaud et al., 2022) augment language models with retrieved documents for factual grounding. Recent work explores adaptive retrieval (Asai et al., 2024), multi-document reasoning (Trivedi et al., 2023), and confidence-aware generation (Kadavath et al., 2022). SupportBot extends RAG to conversational knowledge sources with confidence-tiered retrieval across two collections.

Technical Support Automation. Prior work includes intent classification in support dialogues (Qu et al., 2019, 2018), response selection (Wu et al., 2017), and knowledge-grounded dialogue (Dinan et al., 2019; Kim et al., 2020). Recent systems use LLMs for support automation (Chen et al., 2024), but typically operate over static knowledge bases. SupportBot differs by continuously mining knowledge from the conversations it monitors.

Knowledge Base Construction from Text. Automatic KB construction has been studied extensively (Carlson et al., 2010; Mitchell et al., 2018). Open information extraction (Banko et al., 2007; Angeli et al., 2015) and event extraction (Du and Cardie, 2020) extract structured knowledge from unstructured text. Our case mining can be viewed as domain-specific extraction tailored for support conversations, with the novel addition of confidence tiers and temporal promotion.

Support Dialogue Datasets. The Ubuntu Dialogue Corpus (Lowe et al., 2015) provides IRC-based technical dialogues; MSDialog (Qu et al., 2018) offers annotated Microsoft forum threads; Doc2Dial (Feng et al., 2020) provides document-grounded dialogues. These datasets contain pre-structured threads with clear boundaries. SUPPORTBENCH differs by providing raw group chat streams where conversation boundaries must be discovered, matching the real-world setting of deployed support bots.

LLM-as-Judge. Using language models for evaluation has become standard practice (Zheng et al., 2023; Dubois et al., 2024; Li et al., 2024). We adopt multi-dimensional judging with separate scores for correctness, helpfulness, specificity, and necessity, with Google Search grounding enabling the judge to fact-check technical claims.

7 Discussion

7.1 Dual-RAG vs. Single-Collection RAG

The separation of confirmed and tentative knowledge addresses a fundamental challenge in community support: not all advice is verified. A single-collection RAG treats all mined cases equally, potentially presenting unverified recommendations with the same confidence as confirmed solutions.

The dual-RAG architecture enables the response generator to explicitly hedge on recommendation-sourced information (e.g., "Based on a community suggestion (not yet verified)..."), improving user trust and safety.

7.2 Unified vs. Cascaded Analysis

The unified buffer analysis performs all extraction tasks in a single LLM call, replacing a prior cascade of $1+N+M$ calls. Beyond the obvious latency and cost reduction, this approach improves *coherence*: the model sees the full conversation context when making all decisions, avoiding inconsistencies that arise when separate extractors process overlapping message windows.

7.3 Multimodal Support

Support conversations frequently include screenshots, error photos, and hardware images. The production system processes these multimodally: the unified LLM call receives interleaved text and images, enabling case extraction from visual evidence (e.g., "the LED pattern in the photo indicates a bootloader error"). SUPPORTBENCH preserves media type annotations; future versions may include opt-in image subsets.

7.4 Abstention and Coverage

The full system achieves lower per-query accuracy than chat-only because it attempts to answer a broader range of questions. This reflects a fundamental tradeoff: aggressive abstention maximizes precision but reduces utility. Our current threshold (abstain when retrieval confidence < 0.3) targets $>75\%$ accuracy while maintaining reasonable coverage. Deployments may tune this threshold based on the relative costs of false answers vs. missed questions.

8 Conclusion

We presented SupportBot, a technical support system built on the insight that community conversations are a continuously growing, confidence-stratified knowledge base. The dual-RAG architecture (SCRAG + RCRAG) with promotion tracking separates confirmed solutions from tentative recommendations, enabling confidence-aware response generation. The unified multimodal buffer analysis extracts cases, detects solved arcs, and manages knowledge lifecycle in a single LLM call.

On SUPPORTBENCH, SupportBot achieves a Score of 7.8/10 with 100% precision and 93% recall on Ardupilot-UA, outperforming both a context-stuffing LLM baseline (7.2) and a standard chunked-RAG system (6.3) while using 5× fewer tokens. To enable broader evaluation, we release SUPPORTBENCH, a multilingual benchmark of 60,000 messages from six technical support communities across three languages and six domains.

Future directions include: (1) expanding evaluation across all SUPPORTBENCH datasets and additional domains, (2) ablation experiments isolating SCRAG vs. RCRAG contributions, (3) distributing media files for multimodal evaluation, and (4) longitudinal studies of how dual-RAG performance evolves as case indices grow.

Limitations

- **Production results from single deployment**: Our main quantitative results come from one private community. SUPPORTBENCH enables broader evaluation but full cross-domain experiments are ongoing.
- **LLM-as-judge bias**: Automated evaluation may not fully capture human preferences. We mitigate this with multi-dimensional scoring and manual validation of sampled judgments.
- **Media not distributed**: SUPPORTBENCH includes media type annotations but not binary media files. This limits multimodal evaluation on the public benchmark.
- **Telegram-specific**: All SUPPORTBENCH data comes from Telegram groups. Generalization to other platforms (Discord, Slack, forums) requires further study, though the unified format abstracts platform-specific features.
- **Temporal dynamics**: We do not yet study how case extraction quality and retrieval performance change as the knowledge base grows over deployment timescales.

Ethics Statement

SUPPORTBENCH is constructed from public Telegram groups where messages are visible to anyone. All user identifiers are irreversibly anonymized via SHA-256 hashing before distribution. We do not modify message content. The production evaluation uses data from a private community with appropriate access permissions.

SupportBot includes an abstention mechanism to prevent answering beyond its knowledge boundary.

However, automation bias—where users over-trust bot responses—remains a concern. We recommend deploying with clear bot attribution and easy escalation to human experts.

Data Availability

SUPPORTBENCH (60,000 messages, 6 datasets) will be released under a research-use license at [ANONYMIZED]. The production evaluation dataset from the private community cannot be released due to the sensitive nature of technical discussions.

References

Gabor Angeli, Melvin Jose Johnson Premkumar, and Christopher D Manning. 2015. Leveraging linguistic structure for open domain information extraction. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics*, pages 344–354.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*.

Michele Banko, Michael J Cafarella, Stephen Soderland, Matt Broadhead, and Oren Etzioni. 2007. Open information extraction from the web. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence*, pages 2670–2676.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, and 1 others. 2022. Improving language models by retrieving from trillions of tokens. *International Conference on Machine Learning*, pages 2206–2240.

Andrew Carlson, Justin Betteridge, Bryan Kisiel, Burr Settles, Estevam R Hruschka Jr, and Tom M Mitchell. 2010. Toward an architecture for never-ending language learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 24, pages 1306–1313.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *International Conference on Learning Representations*.

Emily Dinan, Stephen Roller, Kurt Shuster, Angela Fan, Michael Auli, and Jason Weston. 2019. Wizard of Wikipedia: Knowledge-powered conversational agents. In *International Conference on Learning Representations*.

Xinya Du and Claire Cardie. 2020. Event extraction by answering (almost) natural questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 671–683.

Yann Dubois, Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2024. AlpacaFarm: A simulation framework for methods that learn from human feedback. In *Advances in Neural Information Processing Systems*, volume 36.

Song Feng, Hui Wan, Chulaka Gunasekara, Siva Patel, Sachindra Joshi, and Luis Lastras. 2020. doc2dial: A goal-oriented document-grounded dialogue dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 8118–8128.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. REALM: Retrieval-augmented language model pre-training. In *International Conference on Machine Learning*, pages 3929–3938.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Nguyen, and 1 others. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Byeongchang Kim, Jaewoo Ahn, and Gunhee Kim. 2020. Sequential latent knowledge selection for knowledge-grounded dialogue. In *International Conference on Learning Representations*.

Tao Lei, Hrishikesh Joshi, Regina Barzilay, Tommi Jaakkola, Kateryna Tymoshenko, Alessandro Moschitti, and Lluis Marquez. 2016. Semi-supervised question retrieval with gated convolutions. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 1279–1289.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Hanling Li, Pengfei Shao, Beirao Zhu, and 1 others. 2024. Generation meets verification: Accelerating large language model inference with smart parallel auto-correct decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Ryan Lowe, Nissan Pow, Iulian Serban, Sakshi Bhatt, Joelle Pineau, and Laurent Charlin. 2015. The Ubuntu dialogue corpus: A large dataset for research in unstructured multi-turn dialogue systems. In *Proceedings of the 16th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 285–294.

Tom Mitchell, William Cohen, Estevam Hruschka, Partha Talukdar, Bishan Yang, Justin Betteridge, Andrew Carlson, Bhuwan Dalvi, Matt Gardner, Bryan Kisiel, and 1 others. 2018. Never-ending learning. *Communications of the ACM*, 61(5):103–115.

Chen Qu, Liu Yang, W Bruce Croft, Johanne R Trippas, Yongfeng Zhang, and Minghui Qiu. 2019. User intent prediction in information-seeking conversations. In *Proceedings of the 2019 Conference on Human Information Interaction and Retrieval*, pages 25–33.

Chen Qu, Liu Yang, W Bruce Croft, Yongfeng Zhang, Johanne R Trippas, and Minghui Qiu. 2018. Analyzing and characterizing user intent in information-seeking conversations. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, pages 989–992.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037.

Yu Wu, Wei Wu, Chen Xing, Ming Zhou, and Zhoujun Li. 2017. Sequential matching network: A new architecture for multi-turn response selection in retrieval-based chatbots. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 496–505.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, and 1 others. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36.